

Sorting:

Sort: Arranging in a particular order.

Why? ->

Selection sort is a simple and efficient sorting algorithm that works by repeatedly selecting the smallest (or largest) element from the unsorted portion of the list and moving it to the sorted portion of the list.

Ascending: least value will in start of the array

Before Sorting

5	20	9	3	6	8
0	1	2	3	4	5

1. Choose/**select** the value available in **index 0** and compare with all other elements and bring the least in the index 0 position.

5>20

5	20	9	3	6	8	5>20 No swap
0	1	2	3	4	5	

5	20	9	3	6	8	5>9 No swap
0	1	2	3	4	5	

3	20	9	5	6	8	5>3 swap
0	1	2	3	4	5	

3	20	9	5	6	8	3>6 No swap
0	1	2	3	4	5	

3	20	9	5	6	8	3>8 No swap
0	1	2	3	4	5	

2nd step/iteration

now it will pic up **index 1** and compare with rest bring least on in index 1;

3	20	9	5	6	8	20>9 Swap
0	1	2	3	4	5	

3	9	20	5	6	8	9>5 swap
3	5	20	9	6	8	5>6 No swap
3	5	20	9	6	8	5>8 No Swap

3	5	20	9	6	8	20>9 Swap
0	1	2	3	4	5	
3	5	9	20	6	8	
3	5	6	20	9	8	
3	5	6	20	9	8	

3	5	6	20	9	8	
3	5	6	9	20	8	
3	5	6	8	20	9	

3	5	6	8	20	9	
3	5	6	8	9	20	

After Sorting

3	5	6	8	9	20
---	---	---	---	---	----

Complexity Analysis of Selection Sort

Time Complexity: The time complexity of Selection Sort is $O(N^2)$ as there are two nested loops:

- One loop to select an element of Array one by one = $O(N)$
- Another loop to compare that element with every other Array element = $O(N)$
- Therefore overall complexity = $O(N) * O(N) = O(N*N) = O(N^2)$

Auxiliary Space: $O(1)$ as the only extra memory used is for temporary variables while swapping two values in Array. The selection sort never makes more than $O(N)$ swaps and can be useful when memory writing is costly.

Bubble Sort is the simplest [sorting algorithm](#) that works by repeatedly swapping the adjacent elements if they are in the wrong order. This algorithm is not suitable for large data sets as its average and worst-case time complexity is quite high.

Before Sorting

5	20	9	3	6	8
---	----	---	---	---	---

After Sorting

3	5	6	8	9	20
---	---	---	---	---	----

it is going to compare adjacent two elements and sort it in order.

5>20

Before Sorting

5	20	9	3	6	8	5>20 No Swap
5	20	9	3	6	8	20>9 Swap
5	9	20	3	6	8	20>3 Swap
5	9	3	20	6	8	20>6 Swap
5	9	3	6	20	8	20>8 Swap
5	9	3	6	8	20	

2nd iteration last but one will be sorted

5	9	3	6	8	20	5>9 No Swap
5	9	3	6	8	20	
5	3	9	6	8	20	9>3 Swap
5	3	6	9	8	20	9>8 swap
5	3	6	8	9	20	

5	3	6	8	9	20
5	3	6	8	9	20
5	3	6	8	9	20
5	3	6	8	9	20

5	3	6	8	9	20
---	---	---	---	---	----

3	5	6	8	9	20
---	---	---	---	---	----

3	5	6	8	9	20
---	---	---	---	---	----

Complexity Analysis of Bubble Sort:

Time
Auxiliary Space: $O(1)$

Complexity: $O(N^2)$

***Insertion sort** is a simple sorting algorithm that works similar to the way you sort playing cards in your hands. The array is virtually split into a sorted and an unsorted part. Values from the unsorted part are picked and placed at the correct position in the sorted part.*

Before Sorting

5	20	9	3	6	8
---	----	---	---	---	---

After Sorting

3	5	6	8	9	20
---	---	---	---	---	----

Time Complexity: $O(N^2)$
Auxiliary Space: $O(1)$

Time Complexity of Insertion Sort

- The **worst-case** time complexity of the Insertion sort is $O(N^2)$
- The **average case** time complexity of the Insertion sort is $O(N^2)$
- The time complexity of the **best case** is $O(N)$.

Space Complexity of Insertion Sort

The auxiliary space complexity of Insertion Sort is $O(1)$